

Universidad de los Andes

Facultad de Ingeniería — Ingeniería de Sistemas y Computación

ISIS1511 – Fundamentos de Bases de Datos

Wheels Alpes

Entrega 1 — Diseño de la Base de Datos

Integrantes:

Samuel Zuluaga - 202520012

David Cortés - 202421997

Jose Miguel Angel - 202421569

13 de septiembre de 2026

1. Introducción

Wheels Alpes es una plataforma de carpooling pensada para estudiantes universitarios, que busca resolver el problema de la movilidad diaria hacia y desde la universidad de una forma más económica y organizada. La plataforma tiene dos tipos de usuario: el conductor, que registra su vehículo y publica viajes indicando fecha, hora, ruta, paradas y tarifa; el pasajero, que busca viajes disponibles y solicita un upo eligiendo un punto de recogida. El conductor decide si acepta o rechaza cada solicitud, y una vez es aceptada queda confirmada la reserva del pasajero en ese viaje.

Para esta entrega, el objetivo es diseñar la base de datos que soporta toda esta operación. Esto incluye construir el modelo entidad/relación que represente correctamente las entidades de negocio (estudiantes, conductores, pasajeros, vehículos, viajes, paradas, solicitudes) junto con sus atributos y las relaciones entre ellas, incluyendo las cardinalidades y reglas de obligatoriedad que se exigen.

A partir de ese modelo entidad/relación, la entrega también contempla pasar al modelo relaciones siguiendo el proceso visto en clase, revisar en que forma normal queda cada tabla y hacer los ajustes necesarios hasta llevar todo a FNBC. Finalmente, se deben diseñar los escenarios de prueba que permitan validar que el modelo funciona correctamente: pruebas de llaves primarias, de integridad referencial con llaves foráneas, y de las restricciones de chequeo definidas sobre los datos.

2. Análisis y modelo conceptual (E/R)

2.1 Descripción de entidades y relaciones

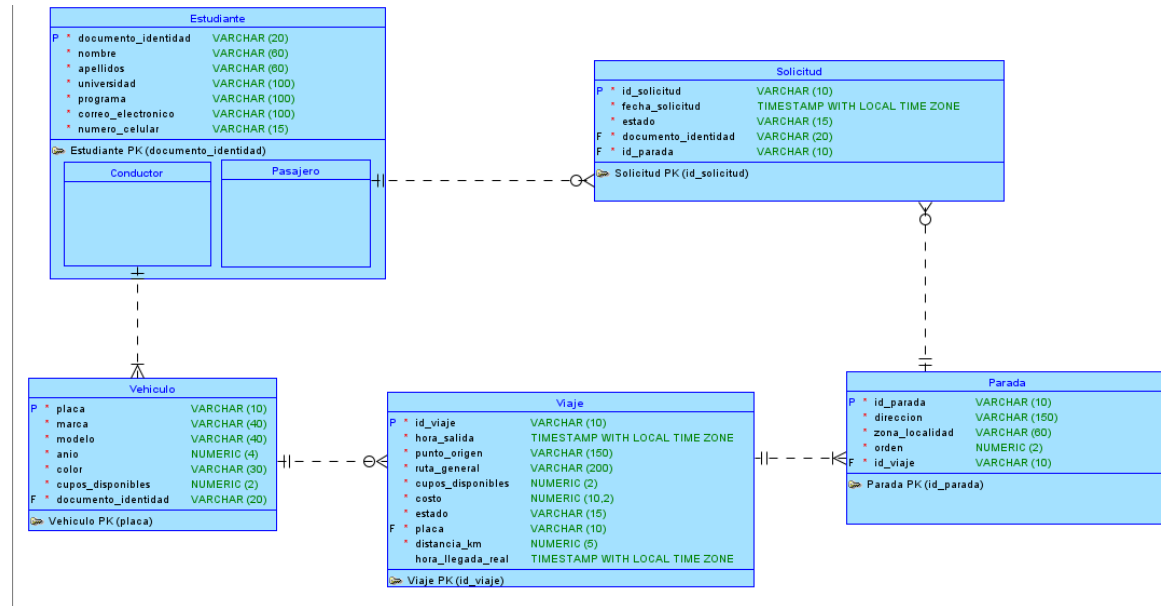
Conductor y Pasajero salen de Estudiante por herencia porque los dos comparten exactamente los mismos datos personales, y en el enunciado un mismo estudiante puede ser conductor, pasajero o ambos. Si se hubieran hecho como entidades separadas sin herencia, se tendría que repetir los mismos 7 atributos en las dos y no se podría representar bien que una persona cumple con los dos roles al mismo tiempo. Por eso la especialización es overlapping: un estudiante puede estar en las dos subclases sin problema.

En cuanto a Solicitud, se hizo como una entidad y no solo una relación entre Pasajero y Viaje porque una solicitud no depende solo de quién la hizo y de qué viaje es, también depende de en qué parada la persona quiere que la recojan. Son tres aspectos relacionados al tiempo y además la Solicitud tiene sus propios datos.

Por otro lado, no existe una relación directa entre Conductor y Viaje porque ese dato ya se puede sacar siguiente Viaje -> Vehículo -> Conductor: como cada vehículo es de un solo conductor y cada viaje usa un solo vehículo, no hace falta guardar el conductor otra vez directamente en Viaje. Si lo dejara, estaría guardando el mismo dato dos veces por dos caminos distintos, lo cual da problemas más adelante.

Por último, Parada es una entidad y no un atributo de Viaje porque un Viaje puede tener varias paradas y porque es necesario referencias la misma parada desde Solicitud, cosa que no podría hacer si fuera solo un atributo dentro de Viaje.

2.2 Diagrama Entidad-Relación



3. Diseño de la base de datos

3.1 Modelo de datos relacional

Explicaciones de las transformaciones de Chen:

ESTUDIANTE

Entidad fuerte -> Tabla

Relacion 1:N -> FK en el lado N

Como estudiante es una entidad fuerte se transforma en una tabla con sus mismos atributos y su identificador se vuelve la llave primaria.

Conductor/Pasajero

Tabla por subtipo con FK-PK al supertipo

Como conductor y pasajero son un roles que un estudiante puede tener de forma independiente, es una especializacion parcial y no disjunta respecto a Estudiante.

VEHICULO

Entidad fuerte -> Tabla

Conductor(1) -- Vehiculo(N)

Para las relaciones 1:N no se crea una tabla nueva sino que propagamos la llave primaria del lado del conductor y con llave primaria en vehiculo, la cual tiene su propia llave primaria (placa) por lo que es una entidad fuerte.

VIAJE

Entidad fuerte -> Tabla

Vehículo(1) -- Viaje(N)

Es la misma relación que la anterior, por lo que se conecta a Vehículo por la llave foránea y la entidad Viaje conserva su llave primaria (id_viaje) por lo que es una entidad fuerte.

PARADA

Entidad fuerte -> Tabla

Viaje(1) -- Parada(N)

Parada es una entidad fuerte con su propia llave primaria (id_parada). La relación con Viaje es obligatoria (1:N) por regla de negocio, un viaje sin paradas no tiene sentido, pero eso no la convierte en entidad débil, ya que su identificador no depende de Viaje. Por eso Viaje.id_viaje se propaga como llave foránea NN en Parada

Solicitud

Entidad fuerte -> Tabla
Pasajero(1) -- Solicitud(N)
Parada(1) -- Solicitud(N)

Solicitud se modela como entidad independiente porque tiene atributos propios (*fechasolicitud, estado*) y dos relaciones 1 a muchos: con Pasajero y con Parada. Como es 1:N, la llave primaria del lado univalor migra como llave foranea al lado N.

A continuación, la ficha de cada relación ():

Estudiante.

documento_identidad	nombre	apellidos	universidad	programa	correo_electronico	numero_celular
PK, NN	NN	NN	NN	NN	NN, ND	NN
1002000001	Sebastian	Acosta Giraldo	Los Andes	Ing. Industrial	s.acosta@uniandes.edu.co	3001111111
1002000002	Valentina	Gomez Ruiz	Los Andes	Ing. de Sistemas	v.gomez@uniandes.edu.co	3002222222

Conductor.

documento_identidad
PK, FK (ESTUDIANTE.documento_identidad), NN, ND
1002000001

Pasajero

documento_identidad
PK, FK (ESTUDIANTE.documento_identidad), NN, ND
1002000002

Vehiculo

placa	marca	modelo	anio	color	cupos_disponibles	documento_identidad
-------	-------	--------	------	-------	-------------------	---------------------

PK, NN, ND	NN	NN	NN	NN	NN	NN, FK (CONDUCTOR.documento_identidad)
ABC123	Chevrolet	Spark	2012	Negro	4	1002000001

Solicitud

id_solicitud	fecha_solicitud	estado	documento_identidad	id_parada
PK, UA	NN	NN, CK[pendiente, aceptada, rechazada, cancelada]	NN, FK (PASAJERO.documento_identidad)	NN, FK (PARADA.id_parada)
S1	13/09/2026 06:00:00	aceptada	1002000002	P1

Viaje

id_viaje	hora_salida	punto_origen	ruta_general	cupos_disponibles	costo	estado	distancia_km	hora_llegada_real	placa
PK, UA	NN	NN	NN	NN	NN	NN, CK[programado, en curso, completado, cancelado]	NN	(vacio hasta completado)	NN, FK (VEHICULO.placa)
V1	13/09/2026 07:00:00	Tv. 3 #78-10	Norte - Uniandes	2	5000	programado	8.5	NULL	ABC123

Parada

id_parada	direccion	zona_localidad	orden	id_viaje
-----------	-----------	----------------	-------	----------

PK, UA	NN	NN	NN	NN, FK (VIAJE.id_viaje)
P1	Calle 1#2-3	Usaquen	1	V1

3.2 Normalización — verificación de formas normales (hasta FNBC)

ESTUDIANTE

1FN	Completo, no hay atributos multivaluados.
2FN	Completo, la PK no es compuesta, entonces no hay posibilidad de dependencias parciales. (documento_identidad es un solo atributo)
3FN	Completo, no hay relaciones transitivas y ningun atributo no-primo determina a otro. (Por ejemplo universidad no determina programa y son independientes dado el estudiante)
FNBC	Completo, documento_identidad es super llave de por sí.
Transformación aplicada	

CONDUCTOR

1FN - FNBC	Completo, no hay atributos multivaluados, porque su PK es simultáneamente su FK, y no hay posibilidad de dependencias parciales, ni transitivas.
2FN	
3FN	
FNBC	
Transformación aplicada	

PASAJERO

1FN-FNBC	Completo, no hay atributos multivaluados, porque su PK es simultáneamente su FK, y no hay posibilidad de dependencias parciales, ni transitivas. Igual que Conductor.
2FN	
3FN	
FNBC	
Transformación aplicada	

VEHICULO

1FN	Completo, todos los atributos son atomicos.
------------	---

2FN	Completo. Creamos PK (placa) sin dependencias parciales.
3FN	Completo No hay atributo no-primo que determine a otro (marca no determina modelo de forma única ya que una marca tiene muchos modelos, y viceversa un modelo podría repetirse entre marcas distintas).
FNBC	Completo. LLave es unica y super llave.
Transformación aplicada	

VIAJE

1FN	Completo. Atributos atómicos; hora_llegada_real puede ser nulo pero no es multivaluado.
2FN	Completo. PK simple (id_viaje).
3FN	Completo. No hay dependencia entre no-primos: placa no determina costo ni distancia_km (dos viajes con el mismo vehículo pueden tener costos y distancias distintas).
FNBC	Completo. Unico determinante: id_viaje.
Transformación aplicada	

PARADA

1FN	Cumple. Atributos atómicos.
2FN	Cumple. PK simple, no hay dependencia parcial posible
3FN	No cumple. zona_localidad es un atributo no-primo que depende transitivamente de direccion, que también es no-primo: id_parada - direccion - zona_localidad. Si dos paradas distintas comparten la misma dirección (ej. un punto de encuentro común como un portal de Transmilenio usado en varios viajes), zona_localidad quedaría repetida innecesariamente, con riesgo de inconsistencia.
FNBC	No cumple (por lo de arriba).
Transformación aplicada	Se descompone en dos relaciones: UBICACION(direccion PK, zona_localidad NN) PARADA(id_parada PK, direccion FK-UBICACION NN, orden NN, id_viaje FK NN)

SOLICITUD

1FN	Completo
2FN	Completo. PK simple.
3FN	Completo, no hay dependencias entre no primos (<i>idparada no determina estado</i> ni documento_identidad).
FNBC	Completo. Superllave id_solicitud
Transformación aplicada	

3.3 Modelo relacional final (post-normalización)

Estudiante.

documento_identidad	nombre	apellidos	universidad	programa	correo_electronico	numero_celular
PK, NN	NN	NN	NN	NN	NN, ND	NN
1002000001	Sebastian	Acosta Giraldo	Los Andes	Ing. Industrial	s.acosta@uniandes.edu.co	3001111111
1002000002	Valentina	Gomez Ruiz	Los Andes	Ing. de Sistemas	v.gomez@uniandes.edu.co	3002222222

Conductor.

documento_identidad
PK, FK (ESTUDIANTE.documento_identidad), NN, ND
1002000001

Pasajero

documento_identidad
PK, FK (ESTUDIANTE.documento_identidad), NN, ND
1002000002

Vehiculo

placa	marca	modelo	anio	color	cupos_disponibles	documento_identidad
PK, NN, ND	NN	NN	NN	NN	NN	NN, FK (CONDUCTOR.documento_identidad)
ABC123	Chevrolet	Spark	2012	Negro	4	1002000001

Solicitud

id_solicitud	fecha_solicitud	estado	documento_identidad	id_parada
PK, UA	NN	NN, CK[pendiente, aceptada, rechazada, cancelada]	NN, FK (PASAJERO.documento_identidad)	NN, FK (PARADA.id_parada)
S1	13/09/2026 06:00:00	aceptada	1002000002	P1

Viaje

id_viaje	hora_salida	punto_origen	ruta_general	cupos_disponibles	costo	estado	distancia_km	hora_llegada_real	placa
PK, UA	NN	NN	NN	NN	NN, CK [Costo > 0]	NN, CK[programado, en curso, completado, cancelado]	NN	(vacío hasta completado)	NN, FK (VEHICULO.placa)
V1	13/09/2026 07:00:00	Tv. 3 #78-10	Norte - Uniandes	2	5000	programado	8.5	NULL	ABC123

Parada (Modificada)

id_parada	direccion	orden	id_viaje
PK, UA	NN, FK (UBICACION. Direccion)	NN	NN, FK (VIAJE.id_viaje)
P1	Calle 1#2-3	1	V1

Ubicacion (Nueva)

direccion	zona_localidad
PK, NN	NN
Calle 1#2-3	Usaquen

4. Escenarios de prueba

4.1 Pruebas de unicidad de tuplas

Lo hicimos usando Viaje (PK: id_viaje).

##Todo lo hicimos en estilo SQL porque eso era lo que pedia el esqueleto, pero pues usando lo que vimos hasta la fecha. (Presentacion Introducción a SQL- Algebra)

Caso 1 – Insertar tupla con PK nueva.

```
INSERT INTO Viaje (id_viaje, hora_salida, punto_origen, ruta_general, cupos_disponibles, costo, estado, distancia_km, hora_llegada_real, placa) VALUES ('V2', '2026-09-14 08:00:00', 'Calle 100 #15-20', 'Norte - Uniandes', 3, 4500, 'programado', 6.20, NULL, 'ABC123');
```

Resultado esperado – 1 fila insertada.

Caso 2- Insertar una segunda tupla con la MISMA PK ('V2') (debe fallar):

```
INSERT INTO Viaje (id_viaje, hora_salida, punto_origen, ruta_general, cupos_disponibles, costo, estado, distancia_km, hora_llegada_real, placa) VALUES ('V2', '2026-09-14 09:00:00', 'Calle 80 #10-5', 'Occidente - Uniandes', 2, 4000, 'programado', 5.10, NULL, 'ABC123');
```

Resultado esperado- Deberia ser rechazado ya que ya existe una fila con esa misma llave primaria.

4.2 Pruebas de integridad referencial (FK)

Usamos una tabla para cada relacion con llave foranea

Relación FK	Dato de entrada	Resultado esperado
Conductor - Estudiante	documento_identidad = '1002000001' (existe)	Insertión exitosa
Conductor -Estudiante	documento_identidad = '9999999999' (no existe)	Rechazado — FK inexistente
Pasajero - Estudiante	documento_identidad = '1002000002' (existe)	Insertión exitosa
Pasajero - Estudiante	documento_identidad = '8888888888' (no existe)	Rechazado — FK inexistente
Vehiculo - Conductor	documento_identidad = '1002000001' (existe)	Insertión exitosa
Vehiculo - Conductor	documento_identidad = '1002000009' (no es conductor)	Rechazado — FK inexistente
Viaje - Vehiculo	placa = 'ABC123' (existe)	Insertión exitosa
Viaje - Vehiculo	placa = 'ZZZ999' (no existe)	Rechazado — FK inexistente
Parada - Viaje	id_viaje = 'V1' (existe)	Insertión exitosa

Parada -Viaje	id_viaje = 'V9' (no existe)	Rechazado — FK inexistente
Parada - Ubicacion	direccion = 'Calle 1#2-3' (existe)	Inserción exitosa
Parada -Ubicacion	direccion = 'Kra 50#1-1' (no existe en Ubicacion)	Rechazado — FK inexistente
Solicitud -Pasajero	documento_identidad = '1002000002' (existe)	Inserción exitosa
Solicitud - Pasajero	documento_identidad = '1002000001' (es conductor, NO pasajero)	Rechazado — FK inexistente
Solicitud - Parada	id_parada = 'P1' (existe)	Inserción exitosa
Solicitud - Parada	id_parada = 'P9' (no existe)	Rechazado — FK inexistente

Solo vamos a mostrar un ejemplo de sentencia, no todos los casos.

“INSERT INTO Solicitud (id_solicitud, fecha_solicitud, estado, documento_identidad, id_parada) VALUES ('S9', '2026-09-13 06:30:00', 'pendiente', '1002000002', 'P9'); -- Resultado esperado: rechazado, 'P9' no existe como id_parada en Parada”

4.3 Pruebas de integridad por restricciones de chequeo (CHECK)

Restricción CK	Dato de entrada	Resultado esperado
Viaje.estado	estado = 'volando'	Rechazado — el valor no está permitido por el CHECK
Viaje.estado	estado = 'completado'	Inserción/actualización exitosa
Solicitud.estado	estado = 'en_proceso'	Rechazado — el valor no está permitido por el CHECK
Solicitud.estado	estado = 'rechazada'	Inserción exitosa
Viaje.costo	costo = -500	Rechazado — el valor no está permitido por el CHECK
Viaje.costo	costo = 5000	Inserción exitosa

Sentencia ejemplo.

INSERT INTO Viaje (id_viaje, hora_salida, punto_origen, ruta_general, cupos_disponibles, costo, estado, distancia_km, hora_llegada_real, placa) VALUES ('V4', '2026-09-14 10:00:00', 'Calle 26', 'Centro - Uniandes', 2, -500, 'programado', 4.0, NULL, 'ABC123');

-- Resultado esperado: rechazado, el costo no puede ser negativo o cero

4.4 Pruebas funcionales por requerimiento (RF1–RF11, RFC1–RFC3, RFA1–RFA2)

Creemos una tabla grande para poder mostrar los diferentes casos para cada requerimiento

Req.	Caso	Rol	Entrada / Consulta	Resultado esperado
RF1	Éxito	Conductor	INSERT en Estudiante con documento_identidad nuevo	Inserción exitosa
RF1	Falla	Conductor	INSERT con documento_identidad ya existente	Rechazado — PK duplicada
RF2	Éxito	Conductor	INSERT en Vehiculo con placa nueva y conductor existente	Inserción exitosa
RF2	Falla	Conductor	INSERT en Vehiculo con FK a un documento que no es Conductor	Rechazado — FK inexistente
RF3	Éxito	Conductor	INSERT en Viaje con vehículo propio + INSERT de sus paradas iniciales	Inserción exitosa
RF3	Falla	Conductor	INSERT en Viaje con placa de un vehículo ajeno	Rechazado a nivel de aplicación (regla de negocio)
RF4	Éxito	Conductor	UPDATE Solicitud SET estado='aceptada' (estaba pendiente)	Actualización exitosa

RF4	Falla	Conductor	UPDATE Solicitud SET estado='aceptada' (ya estaba cancelada)	Rechazado por regla de negocio (transición inválida)
RF5	Éxito	Conductor	UPDATE Viaje SET estado='en curso' (estaba programado)	Actualización exitosa
RF5	Falla	Conductor	UPDATE Viaje SET estado='en curso' (ya estaba completado)	Rechazado por regla de negocio (secuencia inválida)
RF6	Éxito	Conductor	Consultar viajes propios: Π (σ Viaje $\bowtie$ Vehiculo $\bowtie$ Conductor)	Retorna lista de viajes con estado y pasajeros
RF6	Falla	Conductor	Consultar historial de otro conductor sin permiso	Rechazado a nivel de aplicación
RF7	Éxito	Pasajero	INSERT en Estudiante + Pasajero con documento nuevo	Inserción exitosa
RF7	Falla	Pasajero	INSERT con correo_electronico duplicado	Rechazado — ya existe ese correo (ND)
RF8	Éxito	Pasajero	UPDATE Estudiante SET numero_celular = ... (propio)	Actualización exitosa
RF8	Falla	Pasajero	UPDATE Estudiante SET documento_identidad = ... (es PK)	Rechazado a nivel de aplicación (no editable)
RF9	Éxito	Público (sin login)	σ (fecha, zona) (Viaje $\bowtie$ Parada $\bowtie$ Ubicacion)	Retorna viajes disponibles
RF9	Falla	Público (sin login)	Buscar con fecha en formato inválido	Error de validación de entrada
RF10	Éxito	Pasajero autenticado	INSERT en Solicitud con FK válidas a Pasajero y Parada	Inserción exitosa, estado='pendiente'
RF10	Falla	No autenticado	Intentar INSERT en Solicitud sin sesión válida	Rechazado a nivel de aplicación
RF11	Éxito	Pasajero	UPDATE Solicitud SET estado='cancelada' ($\geq 2h$ de anticipación)	Actualización exitosa
RF11	Falla	Pasajero	UPDATE Solicitud SET estado='cancelada' ($< 2h$ de anticipación)	Rechazado por regla de negocio (validación de tiempo)
RFC1	Éxito	Administración	Π zona_localidad (σ fechaErango (Solicitud $\bowtie$ Parada $\bowtie$ Ubicacion)), agrupado y contado por zona	Retorna demanda por localidad
RFC2	Éxito	Administración	Igual que RFC1, pero sumando Viaje.costo de las solicitudes aceptadas por zona	Retorna ingresos estimados por localidad
RFC3	Éxito	Administración	Π fecha,conductor (Viaje $\bowtie$ Vehiculo $\bowtie$ Conductor), agrupado por fecha y conductor	Retorna evolución de viajes por conductor
RFA1	Éxito	Conductor	Sobre σ estado='completado' (Viaje $\bowtie$ Vehiculo del conductor), contar pasajeros y sumar costo y distancia_km	Retorna métricas del tablero de conductor
RFA1	Falla	Conductor	Incluir un viaje con hora_llegada_real = NULL en el cálculo de tiempo	Excluido del cálculo (regla de negocio: solo completados)
RFA2	Éxito	Pasajero	Sobre σ Solicitud.pasajero=X (Solicitud $\bowtie$ Parada $\bowtie$ Viaje), contar viajes y promediar distancia_km y tiempo	Retorna métricas del tablero de pasajero
RFA2	Falla	Pasajero	Calcular zona más frecuente sin ninguna Solicitud aceptada	Sin datos, manejado en aplicación